



Memorial University of Newfoundland

Faculty of Engineering and Applied Science

ENGI-9839

Software Verification and Validation

Manual Testing vs. Automated Verification

A Comparative Adequacy Study Using Mutation Score

Course Project Report

Student Name: MD Abdul Hadi Chowdhury

Student ID: 202583341

Course: ENGI-9839

Instructor: Raja Abbas

Submission Date: July 2026

Abstract

This study compares the adequacy of two software testing approaches applied to the financial transaction subsystem of an open-source Django banking application. The first approach used manually designed pytest test cases based on equivalence partitioning and boundary value analysis. The second approach used property-based testing with Hypothesis to generate inputs from financial requirements and invariants.

Both test suites were evaluated against the same pinned version of the subject system and the same mutation-testing scope. Structural coverage was measured using `pytest-cov`, while fault-detection adequacy was assessed using `mutmut`. The manual suite contained 29 test functions and achieved 84% coverage. The property-based suite contained 16 property test functions and achieved 77% coverage.

The `mutmut` tool generated 96 mutants in each experimental run. Twenty-six mutants belonged to an out-of-scope transaction date-range form and were excluded consistently from the controlled comparison. This left 70 relevant mutants. The manual suite killed 53 relevant mutants, resulting in a controlled mutation score of 75.71%. The property-based suite killed 42 relevant mutants, resulting in a controlled score of 60.00%.

When only mutants reached by each suite were considered, the scores were much closer: 75.71% for the manual suite and 73.68% for the property-based suite. This indicates that the main difference was behavioural reach rather than a large difference in fault-detection strength within executed code. Survivor analysis also identified shared weaknesses in interest arithmetic, currency rounding, and interest-period verification. The findings support using manual EP/BVA testing and property-based testing as complementary techniques.

Contents

Abstract	i
List of Tables	viii
List of Figures	ix
1 Introduction	1
2 Background and Related Work	2
2.1 Equivalence Partitioning	2
2.2 Boundary Value Analysis	3
2.3 Property-Based Testing	3
2.4 Structural Coverage	4
2.5 Mutation Testing	4
2.6 Related Empirical Work	5
3 Subject System and Experimental Scope	6
3.1 Selected Subject System	6
3.2 Original Testing Condition	6
3.3 Selected Financial Subsystem	7
3.4 Excluded Functionality	8
3.5 Financial Rules Observed in the Implementation	9
3.6 Absence of Inter-Account Transfers	9
3.7 Repository Artifact	9
4 Requirements, Invariants, and Test Oracles	10
4.1 Requirements Extraction	10

4.2	Deposit Requirements	10
4.3	Withdrawal Requirements	11
4.4	Transaction Requirements	12
4.5	Interest and Account Requirements	12
4.6	Financial Invariants	12
4.7	Test Oracle Definition	13
4.8	Equivalent and Underspecified Behaviour	14
4.9	Traceability Between Requirements and Test Design	14
5	Experimental Methodology and Environment	15
5.1	Research Design	15
5.2	Experimental Workflow	16
5.3	Software Environment	17
5.4	Test-Suite Organization	17
5.5	Coverage Measurement	18
5.6	Mutation-Testing Procedure	19
5.7	Mutation Outcome Categories	19
5.8	Mutation-Score Definitions	20
5.9	Controlled Mutant Population	20
5.10	Survivor Analysis Procedure	21
5.11	Reproducibility Controls	21
6	Test-Suite Design	22
6.1	Manual EP/BVA Test Suite	22
6.1.1	Deposit Test Design	22
6.1.2	Withdrawal Test Design	23

6.1.3	Interest and Account Tests	23
6.1.4	Strengths and Limitations of the Manual Design	24
6.2	Property-Based Test Suite	24
6.2.1	Input Strategies	25
6.2.2	Deposit Properties	25
6.2.3	Withdrawal Properties	26
6.2.4	Interest Properties	26
6.2.5	Shrinking	27
6.2.6	Property-Catalogue Limitation	27
6.3	Comparison of the Two Designs	27
7	Execution Results and Coverage Comparison	28
7.1	Test Execution Results	28
7.2	Overall Structural Coverage	29
7.3	Per-Module Coverage	29
7.4	Coverage Interpretation	30
7.5	Coverage Evidence Artifacts	30
7.6	Coverage Comparison	31
8	Mutation Testing Results	31
8.1	Generated Mutant Population	31
8.2	Raw Mutation Outcomes	32
8.3	Controlled Relevant Mutant Population	33
8.4	Controlled Mutation Scores	33
8.5	Reached-Mutant Scores	34
8.6	Interpretation of No-Test Outcomes	35

8.7	Mutation Result Artifacts	36
8.8	Mutation Outcome Comparison	36
9	Surviving-Mutant Analysis	37
9.1	Purpose of Survivor Analysis	37
9.2	Survivor Classification	38
9.3	Genuine Test Weaknesses	38
9.3.1	Interest Arithmetic	38
9.3.2	Monetary Rounding	39
9.3.3	Interest-Calculation Scheduling	39
9.3.4	Positional Form Initialization	40
9.4	Equivalent or Likely Equivalent Mutants	40
9.5	Underspecified Diagnostic-Text Mutants	40
9.6	Outside-Scope Survivor	41
9.7	Manual and Property Survivor Relationship	41
9.8	Lessons from Survivor Analysis	42
9.9	Survivor Analysis Artifact	42
10	Discussion	42
10.1	Answer to the Primary Research Question	42
10.2	RQ1: Which Suite Achieved Higher Structural Coverage?	43
10.3	RQ2: Which Suite Reached a Broader Range of Financial Behaviour?	44
10.4	RQ3: Which Categories of Mutants Survived?	44
10.5	RQ4: What Relationship Was Observed Between Coverage and Mutation Adequacy?	44
10.6	Manual EP/BVA Strengths	45
10.7	Manual EP/BVA Limitations	45

10.8	Property-Based Testing Strengths	46
10.9	Property-Based Testing Limitations	46
10.10	Complementary Use of Both Approaches	46
10.11	Implications for Financial Software Testing	47
10.12	Overall Interpretation	48
11	Threats to Validity	48
11.1	Internal Validity	48
11.2	Construct Validity	49
11.3	External Validity	49
11.4	Conclusion Validity	50
11.5	Reliability and Reproducibility	50
11.6	Researcher Bias	51
11.7	Mitigation Summary	51
12	Reproduction Instructions and Practical Artifact	52
12.1	Artifact Overview	52
12.2	Prerequisites	52
12.3	Repository Setup	53
12.4	Executing the Manual Suite	54
12.5	Executing the Property-Based Suite	54
12.6	Collecting Manual-Suite Coverage	54
12.7	Collecting Property-Suite Coverage	55
12.8	Preparing the Mutation Workspace	56
12.9	Executing the Manual Mutation Run	56
12.10	Executing the Property-Based Mutation Run	57

12.11	Inspecting Mutation Results	57
12.12	Exporting Surviving Mutants	57
12.13	Generated Result Artifacts	58
12.14	Observed Experimental Summary	58
12.15	Reproduction Limitations	58
13	Recommendations and Future Work	59
13.1	Recommended Test Improvements	59
13.2	Recommended Combined Strategy	59
13.3	Future Work	60
14	Conclusion	61
A	Execution Evidence	63
A.1	Manual Test-Suite Evidence	63
A.2	Property-Based Test-Suite Evidence	63
A.3	Coverage Evidence	63
A.4	Mutation-Testing Evidence	64

List of Tables

1	Source modules included in the verification study	8
2	Requirement categories used in the study	10
3	Principal financial invariants	13
4	Examples of requirements-to-test-design traceability	15
5	Experimental software environment	17
6	Manual deposit equivalence partitions	22
7	Manual withdrawal equivalence partitions	23
8	Comparison of the two test-design approaches	28
9	Test-suite execution summary	29
10	Overall structural coverage comparison	29
11	Per-module structural coverage comparison	29
12	Mutation-testing outcome summary	32
13	Controlled mutation-score comparison	34
14	Reached-mutant mutation-score comparison	35
15	Classification of unique surviving mutants	38
16	Principal reproducibility artifacts	58
17	Expected reproduction results	58

List of Figures

1	Structural coverage achieved by the manual and property-based test suites for each selected production module.	31
2	Comparison of controlled relevant-population and reached-mutant mutation scores.	35
3	Mutation-testing outcomes for the manual and property-based test suites. . .	37
4	Manual EP/BVA test-suite execution showing 29 passing tests.	63
5	Property-based test-suite execution showing 16 passing tests.	63
6	Manual-suite statement and branch coverage output.	63
7	Property-based statement and branch coverage output.	64
8	Manual-suite mutation-testing summary.	64
9	Property-based mutation-testing summary.	65

1 Introduction

Software testing is a central activity in software verification and validation because it provides evidence that an implementation behaves according to its requirements. In practice, test suites are often created manually using selected input-output examples. Systematic black-box techniques such as equivalence partitioning and boundary value analysis help testers choose representative cases from valid, invalid, and boundary regions of the input domain.

Property-based testing follows a different approach. Instead of defining only individual examples, the tester expresses general properties that must hold for many automatically generated inputs. In Python, the Hypothesis framework supports this style of testing through reusable data strategies, generated examples, shrinking, and repeated execution of behavioural properties.

Although both approaches can reveal faults, comparing them using only the number of test functions or code coverage can be misleading. A test suite may execute a line of code without checking its result strongly enough to detect an error. Therefore, structural coverage alone does not fully measure fault-detection adequacy.

Mutation testing provides a stronger evaluation mechanism. It introduces small syntactic changes into the production code, such as replacing arithmetic operators, altering constants, or removing function arguments. A mutant is considered killed when at least one test fails against the modified program. If all tests continue to pass, the mutant survives. Mutation score therefore evaluates both execution reach and assertion strength.

In this report, the term *manual suite* refers to manual test-case design, not manual test execution. Both suites were executed automatically using `pytest`. The comparison concerns manually selected EP/BVA examples versus automatically generated Hypothesis inputs and properties.

This project compares a manually designed EP/BVA test suite with a Hypothesis property-based test suite for the financial transaction subsystem of an open-source Django banking application. The selected subsystem includes deposit validation, withdrawal validation, account balance changes, transaction-record creation, interest calculation, and selected account-model behaviour.

The primary research question is:

For the selected financial transaction subsystem, how does the mutation adequacy

of a manually designed equivalence-partitioning and boundary-value test suite compare with that of a Hypothesis property-based test suite?

The project also addresses the following secondary questions:

1. Which suite achieves higher structural coverage?
2. Which suite reaches a broader range of financial behaviours?
3. Which categories of mutants survive both test suites?
4. What relationship is observed between structural coverage and mutation adequacy?

The experiment was controlled by using the same subject-system revision, Python environment, source files, mutation tool, mutation configuration, and target modules. Only the selected test suite changed between the two mutation runs.

The main finding is that the manually designed suite achieved the higher controlled mutation score, primarily because it exercised a broader range of account and transaction behaviour. However, the two suites achieved similar scores when only reached mutants were considered. This suggests that the property-based suite was nearly as effective within the behaviours it tested, but its property catalogue covered a narrower part of the selected subsystem.

The remainder of this report presents the relevant testing background, describes the subject system and experimental design, explains both test suites, reports the coverage and mutation results, analyzes surviving mutants, and discusses the findings, threats to validity, recommendations, and conclusion.

2 Background and Related Work

2.1 Equivalence Partitioning

Equivalence partitioning is a specification-based testing technique that divides an input domain into classes whose members are expected to produce similar behaviour. Instead of testing every possible value, the tester selects representative values from valid and invalid partitions.

For the selected banking subsystem, relevant partitions include valid and invalid deposit amounts, valid and invalid withdrawal amounts, withdrawal amounts below or above configured

limits, and withdrawals that are within or greater than the available balance.

The main benefit of equivalence partitioning is that it reduces a potentially large input domain to a manageable set of meaningful test cases. Its effectiveness depends on whether the tester identifies all important partitions and selects appropriate representatives.

2.2 Boundary Value Analysis

Boundary value analysis focuses on values at, immediately below, and immediately above important thresholds. Many software defects occur near boundaries because of incorrect relational operators, off-by-one errors, or incomplete handling of limiting cases.

Financial applications contain several important boundaries. In this project, the most relevant examples were:

- the minimum accepted deposit amount;
- the minimum withdrawal amount;
- the maximum withdrawal amount; and
- the current account balance.

The manually designed suite therefore included cases below, at, and above these thresholds wherever applicable.

2.3 Property-Based Testing

Property-based testing expresses expected behaviour as general rules rather than only as fixed input-output examples. A property is evaluated repeatedly using automatically generated inputs. QuickCheck popularized this approach by combining generators, executable properties, and shrinking of failing examples [1].

The Python framework Hypothesis follows this model. Test data is generated from reusable strategies, and each property test may execute many examples. When a failure is found, Hypothesis attempts to reduce it to a simpler counterexample.

Property-based testing is especially useful when a requirement can be expressed as an invariant. Examples from the selected banking subsystem include:

- an accepted deposit must not reduce the account balance;
- an accepted withdrawal must reduce the balance by exactly the withdrawal amount;
- an invalid transaction must not modify financial state; and
- interest should remain non-negative for valid positive inputs.

However, automatic input generation cannot compensate for a missing property. Behaviour that is not represented in the property catalogue will not be tested, regardless of how many values are generated.

2.4 Structural Coverage

Structural coverage measures which parts of the implementation are executed by a test suite. Common metrics include statement coverage and branch coverage.

Coverage is useful for identifying unexecuted code, but it does not directly measure whether the tests can detect incorrect behaviour. A statement may be executed without any assertion checking its result. Therefore, a high coverage percentage does not necessarily imply a strong test oracle.

In this project, coverage was used as a secondary adequacy measure. The primary comparison was based on mutation testing.

2.5 Mutation Testing

Mutation testing introduces small syntactic modifications into production code. Each modified version is called a mutant. Typical mutation operators replace arithmetic operators, alter constants, remove arguments, or change return expressions.

A mutant is classified as killed when at least one test fails against it. A mutant survives when all tests still pass. Surviving mutants may indicate weak assertions, incomplete input selection, underspecified behaviour, or semantic equivalence with the original program.

Mutation testing has been widely studied as a fault-based test adequacy criterion [2]. It evaluates both whether code is reached and whether the test oracle is capable of detecting a behavioural change.

A conventional mutation score may exclude confirmed equivalent mutants:

$$MS = \frac{K}{M - E} \times 100, \quad (1)$$

where:

- K is the number of killed mutants;
- M is the total number of generated mutants; and
- E is the number of confirmed equivalent mutants.

Equivalent mutants are syntactically different from the original program but produce the same observable behaviour for all relevant inputs. Because they cannot be distinguished from the original program through observable test outcomes, confirmed equivalent mutants may be excluded from a conventional mutation score.

In this study, the two mutants classified as equivalent or likely equivalent were not removed from the reported scores because their equivalence was not formally proven. Instead, the study reported a separate controlled relevant-population score that consistently excluded 26 mutants belonging to the declared out-of-scope transaction date-range functionality.

Mutation score should not be interpreted as a perfect substitute for real fault detection. Prior empirical research has shown that test-suite size and other confounding factors may influence observed relationships between mutation score and real defects [3].

2.6 Related Empirical Work

Recent empirical research has examined property-based testing in Python projects and evaluated its effectiveness using mutation analysis [4]. Such work suggests that the effectiveness of property-based testing depends strongly on the quality and scope of the properties being tested.

This project extends that line of inquiry through a controlled case study of a financial Django subsystem. The study compares two independently designed test artifacts against the same source revision, mutation targets, and mutation configuration.

3 Subject System and Experimental Scope

3.1 Selected Subject System

The subject system selected for this study was the open-source `saadm11/banking-system` project [5]. The application is implemented in Python using the Django web framework and provides banking-related functionality such as account management, deposits, withdrawals, transaction records, interest calculation, and transaction reporting.

The project was selected because it contains realistic financial rules and state-changing operations suitable for software verification and testing. Financial software is especially appropriate for this study because small arithmetic, validation, or boundary errors may lead to incorrect balances or invalid transactions.

The upstream repository was imported into the project under:

```
subject_system/banking_system/
```

To ensure reproducibility, the evaluated source was pinned to the following upstream commit:

```
9c3ee3eea75030280405506c62d968d646981c05
```

The source repository and pinned revision were documented in `subject_system/UPSTREAM.md`.

3.2 Original Testing Condition

The original project contained test modules under the account and transaction applications. However, these modules did not contain executable test cases. Running the original Django test command collected zero tests.

Therefore, the two test suites evaluated in this study were developed specifically for the experiment:

1. a manually designed pytest suite based on equivalence partitioning and boundary value analysis; and
2. a Hypothesis property-based test suite derived from financial requirements and invariants.

No existing executable tests were reused in either suite.

3.3 Selected Financial Subsystem

The full banking application contains functionality beyond the needs of this study. The experiment therefore focused on a limited financial transaction subsystem.

The selected behaviour included:

- deposit amount validation;
- minimum withdrawal validation;
- maximum withdrawal validation;
- rejection of withdrawals exceeding the available balance;
- account balance increases after accepted deposits;
- account balance decreases after accepted withdrawals;
- preservation of account state after rejected operations;
- creation of transaction records;
- association of transaction records with the correct account;
- storage of the balance after a transaction;
- interest calculation;
- interest-calculation scheduling; and
- selected account-model helper behaviour.

The principal source modules examined were:

Table 1: Source modules included in the verification study

Module	Relevant responsibility
<code>accounts/models.py</code>	Bank-account types, interest calculation, account balances, and interest-calculation month scheduling.
<code>transactions/forms.py</code>	Deposit and withdrawal validation, transaction-form initialization, and transaction-record preparation.
<code>transactions/models.py</code>	Transaction data, transaction type, amount, account association, and balance after transaction.
<code>transactions/views.py</code>	Deposit and withdrawal processing, balance updates, and transaction completion.

3.4 Excluded Functionality

The following functionality was excluded from the experimental scope:

- user authentication and authorization;
- account registration;
- administrative interfaces;
- HTML templates and visual presentation;
- Celery background tasks;
- Redis integration;
- email notification behaviour;
- transaction report rendering;
- transaction date-range filtering; and
- deployment and production infrastructure.

These exclusions kept the experiment focused on financial correctness rather than user-interface, infrastructure, or reporting concerns.

3.5 Financial Rules Observed in the Implementation

The selected implementation enforces several important financial rules.

A deposit must satisfy the configured minimum-deposit amount. Deposits below this threshold are rejected. An accepted deposit increases the account balance by the deposited amount.

A withdrawal must satisfy three conditions:

1. it must not be below the configured minimum withdrawal;
2. it must not exceed the configured maximum withdrawal; and
3. it must not exceed the current account balance.

An accepted withdrawal reduces the account balance by the requested amount. A rejected withdrawal must not modify the balance or create an accepted transaction record.

The implementation also calculates interest using the principal balance, configured annual interest rate, and number of interest calculations per year. The result is rounded to two decimal places.

3.6 Absence of Inter-Account Transfers

The selected system does not implement a direct transfer operation between two accounts. Therefore, a traditional inter-account balance-conservation property could not be tested.

Instead, the study evaluated related financial-state invariants:

- accepted deposits increase the balance by exactly the deposited amount;
- accepted withdrawals decrease the balance by exactly the withdrawn amount; and
- rejected financial operations preserve the original balance.

This adaptation maintained the financial-verification objective without inventing functionality that was not present in the subject system.

3.7 Repository Artifact

The complete project artifact is maintained at:

<https://github.com/hadiChowdhury/Svv-banking-verification-study>

The repository contains the imported subject source, test suites, requirement documents, invariant catalogue, coverage reports, mutation results, surviving-mutant details, and scripts required to reproduce the experiment.

4 Requirements, Invariants, and Test Oracles

4.1 Requirements Extraction

The selected subject system did not provide a complete formal requirements document for financial transaction behaviour. Therefore, the experiment first reconstructed the relevant behavioural requirements from the source code, model configuration, form validation logic, view processing, and observable transaction outcomes.

The resulting specification was documented in:

`docs/requirements_specification.md`

A total of 22 requirements were identified and grouped into the following categories:

Table 2: Requirement categories used in the study

Category	Identifier Range	Count
Deposit requirements	D1–D5	5
Withdrawal requirements	W1–W8	8
Transaction requirements	T1–T4	4
Interest and account requirements	I1–I5	5
Total		22

The requirements were written in terms of externally observable behaviour wherever possible. This reduced dependence on implementation-specific details and supported both manual and property-based test design.

4.2 Deposit Requirements

The deposit requirements were summarized as follows:

- D1:** A deposit below the configured minimum must be rejected.
- D2:** A deposit equal to the configured minimum must be accepted.
- D3:** A deposit greater than the configured minimum must be accepted.
- D4:** An accepted deposit must increase the account balance by exactly the deposited amount.
- D5:** A rejected deposit must not modify the account balance or create an accepted transaction record.

These requirements support both fixed boundary tests and generalized properties over generated deposit amounts.

4.3 Withdrawal Requirements

The withdrawal requirements were summarized as follows:

- W1:** A withdrawal below the configured minimum must be rejected.
- W2:** A withdrawal equal to the configured minimum must be accepted when all other conditions are satisfied.
- W3:** A withdrawal below or equal to the configured maximum may be accepted when sufficient funds are available.
- W4:** A withdrawal greater than the configured maximum must be rejected.
- W5:** A withdrawal greater than the current balance must be rejected.
- W6:** A withdrawal equal to the current balance may be accepted when it satisfies the configured limits.
- W7:** An accepted withdrawal must reduce the balance by exactly the withdrawal amount.
- W8:** A rejected withdrawal must preserve the original balance and must not produce an accepted transaction.

These rules define three independent validation dimensions: minimum amount, maximum amount, and available balance.

4.4 Transaction Requirements

The transaction-record requirements were summarized as follows:

- T1:** An accepted operation must create a transaction record.
- T2:** The transaction record must reference the correct bank account.
- T3:** The transaction amount and type must correspond to the completed operation.
- T4:** The recorded balance after the transaction must equal the actual resulting account balance.

These requirements were evaluated using database assertions and model-level checks.

4.5 Interest and Account Requirements

The account and interest requirements were summarized as follows:

- I1:** Interest must be calculated from the principal balance, annual interest rate, and configured calculation frequency.
- I2:** Interest must be rounded to two decimal places.
- I3:** For positive balances and non-negative rates, calculated interest must not be negative.
- I4:** Increasing the principal balance must not reduce calculated interest when all other parameters remain unchanged.
- I5:** Interest-calculation months must follow the account type's configured calculation frequency.

The fifth requirement became important during mutation analysis because the property-based suite did not directly exercise the corresponding account helper method.

4.6 Financial Invariants

In addition to individual requirements, 13 invariants were documented in:

`docs/invariants.md`

Eight representative invariants are summarized in Table 3; the complete catalogue contains 13.

Table 3: Principal financial invariants

ID	Invariant
INV-1	For an accepted deposit, $B_{\text{after}} = B_{\text{before}} + A$ where A is the deposited amount.
INV-2	For an accepted withdrawal, $B_{\text{after}} = B_{\text{before}} - A$ where A is the withdrawn amount.
INV-3	A rejected deposit or withdrawal must preserve the original balance: $B_{\text{after}} = B_{\text{before}}$
INV-4	An accepted withdrawal must satisfy: $A \leq B_{\text{before}}$
INV-5	A transaction record’s stored balance must equal the actual resulting account balance.
INV-6	For valid positive inputs, calculated interest must be non-negative.
INV-7	For fixed rate and frequency, increasing the principal must not decrease interest.
INV-8	Interest results must use two-decimal monetary precision.

The remaining invariants addressed validation boundaries, transaction associations, scheduling behaviour, and record consistency.

4.7 Test Oracle Definition

A test oracle determines whether observed behaviour is correct. In this study, the oracles were derived from the reconstructed requirements and invariants.

The principal oracle types were:

- **Validation oracle:** determines whether an input is accepted or rejected.

- **Arithmetic oracle:** compares the actual balance or interest with an independently calculated expected value.
- **State-preservation oracle:** verifies that rejected operations do not modify the account or create an accepted transaction.
- **Persistence oracle:** verifies transaction-record fields in the database.
- **Relational oracle:** verifies general relations such as monotonicity or exact balance change.

For example, an accepted withdrawal was not considered correct merely because the form was valid. The test also verified the exact post-operation balance and relevant transaction record.

4.8 Equivalent and Underspecified Behaviour

Some implementation details were intentionally not treated as core financial oracles. In particular, exact wording and capitalization of validation messages were not specified as financial requirements.

Therefore, tests verified that invalid inputs produced validation failure, but they did not generally require an exact error string. This decision later explained several surviving mutants that altered only diagnostic text while preserving rejection behaviour.

Similarly, visual widget configuration and HTML presentation were excluded from the oracle because the study focused on financial correctness rather than user-interface behaviour.

4.9 Traceability Between Requirements and Test Design

The requirements and invariants were used as the common behavioural foundation for both test suites. The manual suite translated them into equivalence partitions and boundary values, while the property-based suite translated them into generated input domains and executable properties.

Table 4: Examples of requirements-to-test-design traceability

ID	Behaviour	Manual design	Property-based design
D1	Deposit below the minimum is rejected.	Invalid equivalence class and value immediately below the threshold.	Generated decimal values below the configured minimum.
D4	Accepted deposit increases balance exactly.	Fixed expected before-and-after balance assertion.	Generated deposit monotonicity and exact balance relation.
W1	Withdrawal below the minimum is rejected.	Lower invalid partition and boundary-adjacent values.	Generated withdrawals below the minimum.
W4	Withdrawal above the maximum is rejected.	Exact maximum and immediately above maximum cases.	Generated values greater than the maximum.
W5	Withdrawal exceeding balance is rejected.	Amounts equal to and greater than the available balance.	Generated amount-balance combinations where amount exceeds balance.
W7	Accepted withdrawal decreases balance exactly.	Fixed expected-value examples.	Generated exact subtraction property.
T2	Transaction record references the correct account.	Database association assertion.	Generated transaction-model association property.
I1	Interest follows the implemented financial formula.	Fixed arithmetic examples.	Generated rate, principal, and frequency properties.
I5	Calculation months follow configured frequency.	Explicit expected month-list examples.	Not directly represented in the original property catalogue.

Table 4 shows that both techniques addressed the same core financial specification, but not every behaviour was represented equally in both suites. This difference became significant when interpreting the final mutation results.

5 Experimental Methodology and Environment

5.1 Research Design

The project used a controlled comparative case-study design. Two independently constructed test suites were evaluated against the same subject-system revision:

1. a manually designed pytest suite based on equivalence partitioning and boundary value analysis; and
2. a property-based suite implemented using Hypothesis.

The independent variable was the test-design approach. The principal dependent variables were:

- total structural coverage;
- per-module structural coverage;
- number of killed mutants;
- number of surviving mutants;
- number of mutants not reached by the suite; and
- mutation score.

To reduce confounding factors, both suites used the same source revision, runtime environment, database configuration, selected production modules, mutation operators, mutation targets, and mutation configuration.

5.2 Experimental Workflow

The experiment followed the workflow below:

1. pin and import the selected banking-system revision;
2. identify the financial transaction scope;
3. extract behavioural requirements and invariants;
4. design the manual EP/BVA test suite;
5. design the Hypothesis property catalogue;
6. execute each suite independently;
7. collect statement and branch coverage;
8. generate mutants from the same pinned source revision and mutation configuration;
9. execute mutation testing separately for each suite;
10. export surviving-mutant details;
11. classify survivors; and

12. compare the two approaches.

The separation between the two suites was preserved throughout the experiment. The manual suite was not executed during the property-based mutation run, and the property-based suite was not executed during the manual mutation run.

5.3 Software Environment

The experiment was executed using the software versions shown in Table 5.

Table 5: Experimental software environment

Component	Version or configuration
Operating environment	macOS local development environment using a Python virtual environment.
Python	Python 3.10.20.
Django	Django 3.2.9.
pytest	pytest 9.1.1.
pytest-django	pytest-django 4.12.0.
pytest-cov	pytest-cov 7.1.0.
Hypothesis	Hypothesis 6.156.6.
mutmut	mutmut 3.6.0.
Database	Django test database created and removed automatically during test execution.
Subject revision	9c3ee3e (full revision recorded in <code>subject_system/UPSTREAM.md</code>).

Exact package dependencies were recorded in the project environment files so that the experiment could be reproduced using the same toolchain.

5.4 Test-Suite Organization

The two suites were stored separately to avoid accidental mixed execution.

The manually designed tests were stored under:

`tests/manual/`

The property-based tests were stored under:

`tests/property_based/`

Each suite was executed using an explicit pytest path. This ensured that only the intended suite contributed to its corresponding coverage and mutation results.

5.5 Coverage Measurement

Structural coverage was measured using `pytest-cov`. Both statement and branch coverage were enabled for the selected source modules.

The coverage scope was limited to:

```
accounts/models.py
transactions/forms.py
transactions/models.py
transactions/views.py
```

For each test suite, the experiment recorded:

- the number of statements;
- the number of missed statements;
- the number of branches;
- the number of partially covered branches; and
- the resulting percentage.

Coverage results were exported in both human-readable and JSON formats:

```
results/coverage/manual_coverage.txt
results/coverage/manual_coverage.json
results/coverage/property_coverage.txt
results/coverage/property_coverage.json
```

Using identical source modules for both runs made the percentages directly comparable.

5.6 Mutation-Testing Procedure

Mutation testing was performed using `mutmut`. The mutation scope was kept identical for both suites.

The procedure for each suite was:

1. prepare a clean mutation workspace;
2. configure the selected test command;
3. generate the production-code mutants;
4. run the selected suite against every mutant;
5. export the complete result list;
6. export the summary;
7. export the surviving mutants; and
8. inspect survivor source changes.

The automation scripts used for this process were:

```
scripts/prepare_mutation_workspace.sh  
scripts/export_surviving_mutants.sh
```

The manual and property-based runs each generated 96 mutants from the same pinned source revision, mutation targets, and configuration. The matching counts provided additional evidence that the mutation population remained stable between the two runs.

5.7 Mutation Outcome Categories

Mutmut classified generated mutants into the following practical categories:

- **Killed:** at least one test failed against the mutant.
- **Survived:** the suite executed successfully and did not detect the mutant.
- **No tests:** the mutated code was not reached by the selected suite.

- **Timeout:** execution exceeded the allowed runtime.

No timeout mutants were observed in either experimental run.

5.8 Mutation-Score Definitions

Two mutation-score interpretations were used.

The first was the controlled relevant-population score:

$$MS_{\text{controlled}} = \frac{K}{M_{\text{relevant}}} \times 100. \quad (2)$$

where K is the number of killed relevant mutants and M_{relevant} is the controlled population after consistently excluding known out-of-scope mutants.

The second was the reached-mutant score:

$$MS_{\text{reached}} = \frac{K}{K + S} \times 100. \quad (3)$$

where S is the number of surviving mutants.

The reached-mutant score excludes mutants classified as no tests. It therefore helps distinguish assertion effectiveness from behavioural reach.

5.9 Controlled Mutant Population

Of the 96 generated mutants, 26 were located in `TransactionDateRangeForm.clean_daterange`. Transaction date-range filtering was excluded from the declared financial scope before mutation testing.

These same 26 mutants were classified as no tests in both runs. They were therefore removed consistently from the controlled comparison:

$$M_{\text{relevant}} = 96 - 26 = 70 \quad (4)$$

This exclusion was applied symmetrically to both suites and did not favour either approach.

5.10 Survivor Analysis Procedure

Surviving mutants were not treated only as numerical outcomes. Each unique manual survivor was inspected and classified according to the observed change and its relationship to the study specification.

The classification categories were:

- genuine test weakness;
- equivalent or likely equivalent mutant;
- underspecified diagnostic-text behaviour; and
- behaviour outside the core study scope.

The analysis considered whether the mutant changed financial behaviour, whether the existing oracle could observe the change, and whether the relevant behaviour was part of the declared requirements.

The detailed analysis was documented in:

`docs/surviving_mutant_analysis.md`

5.11 Reproducibility Controls

The following controls were used to improve reproducibility:

- a pinned upstream source revision;
- explicit dependency versions;
- separate test-suite directories;
- identical coverage targets;
- identical mutation targets;
- shell scripts for workspace preparation and result export;
- preserved raw result files; and
- documented execution commands.

The complete result artifacts were retained under the `results/coverage` and `results/mutation` directories.

6 Test-Suite Design

6.1 Manual EP/BVA Test Suite

The manual suite was designed using equivalence partitioning and boundary value analysis. The objective was to select a small but systematic set of representative examples from valid, invalid, and boundary regions of the financial input domain.

The suite contained 29 pytest test functions. These tests covered validation, state transitions, database persistence, interest calculations, and selected account-model behaviour.

6.1.1 Deposit Test Design

Deposit inputs were divided into the following equivalence classes:

Table 6: Manual deposit equivalence partitions

Partition	Input condition	Expected result	Representative selection
DP-1	Amount below the configured minimum.	Deposit rejected and balance unchanged.	Value immediately below the minimum.
DP-2	Amount equal to the configured minimum.	Deposit accepted.	Exact minimum value.
DP-3	Amount greater than the configured minimum.	Deposit accepted and balance increased exactly.	Typical valid amount.

Boundary value analysis emphasized values immediately below, exactly at, and above the configured minimum deposit threshold.

For accepted deposits, the oracle verified:

$$B_{\text{after}} = B_{\text{before}} + A$$

The tests also checked that the resulting transaction record contained the correct amount, transaction type, account association, and post-transaction balance.

6.1.2 Withdrawal Test Design

Withdrawal behaviour required several interacting partitions because validity depends on the configured minimum, configured maximum, and current account balance.

Table 7: Manual withdrawal equivalence partitions

Partition	Input condition	Expected result	Representative selection
WP-1	Amount below the minimum withdrawal.	Rejected; balance preserved.	Immediately below minimum.
WP-2	Amount equal to the minimum and within the balance.	Accepted.	Exact minimum.
WP-3	Amount between minimum and maximum and within the balance.	Accepted; exact balance reduction.	Typical valid amount.
WP-4	Amount equal to the maximum and within the balance.	Accepted.	Exact maximum.
WP-5	Amount greater than the maximum.	Rejected; balance preserved.	Immediately above maximum.
WP-6	Amount greater than the available balance.	Rejected; balance preserved.	Balance plus a small increment.
WP-7	Amount equal to the available balance and within configured limits.	Accepted; resulting balance becomes zero.	Exact current balance.

For accepted withdrawals, the principal arithmetic oracle was:

$$B_{\text{after}} = B_{\text{before}} - A$$

For rejected withdrawals, the suite checked that the original balance remained unchanged and that no accepted transaction was persisted.

6.1.3 Interest and Account Tests

The manual suite also included explicit examples for:

- interest arithmetic;

- two-decimal rounding;
- non-negative interest results;
- interest-calculation month scheduling;
- account-type behaviour; and
- transaction-model field consistency.

Fixed expected values were calculated independently and compared with the actual results. Explicit examples were also used to validate month lists for the configured interest-calculation frequency.

6.1.4 Strengths and Limitations of the Manual Design

The manual suite provided broad behavioural coverage because the test catalogue explicitly included multiple source-level and model-level responsibilities.

Its main limitation was dependence on tester-selected examples. An important input region could remain untested if it was not identified during partition or boundary analysis.

6.2 Property-Based Test Suite

The property-based suite was implemented using Hypothesis and contained 16 test functions. Each function represented a general property and was executed over multiple generated examples.

Unlike the manual suite, the main test artifact was not a fixed list of values. It consisted of:

- input-generation strategies;
- assumptions and validity constraints;
- financial invariants;
- database setup; and
- relational or arithmetic oracles.

6.2.1 Input Strategies

The generated domains included decimal transaction amounts, account balances, interest rates, and calculation frequencies.

Generated monetary values were constrained to suitable ranges and decimal precision so that the examples remained meaningful for the Django financial models.

Typical strategy categories included:

- deposit values below the minimum;
- valid deposit values at or above the minimum;
- withdrawal values below the minimum;
- valid withdrawals within configured limits;
- withdrawals exceeding the maximum;
- withdrawals exceeding available funds;
- positive account balances;
- non-negative interest rates; and
- valid interest-calculation frequencies.

6.2.2 Deposit Properties

The main deposit properties were:

1. generated values below the minimum are rejected;
2. accepted deposits never reduce the balance;
3. accepted deposits increase the balance by exactly the generated amount; and
4. rejected deposits preserve financial state.

The exact balance-change property was expressed conceptually as:

$$\forall A \in D_{\text{valid}}, \quad B_{\text{after}} = B_{\text{before}} + A$$

where D_{valid} is the generated domain of accepted deposit amounts.

6.2.3 Withdrawal Properties

The withdrawal properties included:

1. generated values below the minimum are rejected;
2. generated values above the maximum are rejected;
3. withdrawals exceeding the available balance are rejected;
4. accepted withdrawals reduce the balance exactly; and
5. rejected withdrawals preserve the original balance.

The accepted-withdrawal property was expressed as:

$$\forall A \in W_{\text{valid}}, \quad B_{\text{after}} = B_{\text{before}} - A$$

The suite generated combinations of transaction amount and account balance to exercise both valid and invalid relationships.

6.2.4 Interest Properties

The property suite tested general relations such as:

- calculated interest is non-negative for valid positive inputs;
- increasing principal does not decrease interest when rate and frequency remain fixed;
- the result follows the implemented arithmetic relation; and
- generated outputs use monetary precision.

A monotonicity property can be expressed as:

$$P_1 \leq P_2 \implies I(P_1, r, n) \leq I(P_2, r, n)$$

where P is principal, r is the annual interest rate, and n is the number of calculations per year.

6.2.5 Shrinking

When Hypothesis finds a failing example, it attempts to reduce that example to a simpler counterexample. This process is known as shrinking.

Shrinking can improve diagnosis because the final reported input is often smaller and easier to understand than the first failing generated value. However, shrinking improves failure explanation only after a relevant property has been defined. It does not reveal behaviour that is absent from the property catalogue.

6.2.6 Property-Catalogue Limitation

The property suite covered the principal deposit, withdrawal, transaction, and interest invariants. However, it did not directly test every selected helper method.

In particular, the original property catalogue did not include a dedicated property for interest-calculation month scheduling. This omission contributed to differences in code reach and mutation outcomes between the two suites.

6.3 Comparison of the Two Designs

Table 8 summarizes the principal differences.

Table 8: Comparison of the two test-design approaches

Dimension	Manual EP/BVA suite	Property-based suite
Test functions	29 fixed pytest test functions.	16 Hypothesis property test functions.
Input selection	Tester-selected representatives and boundary values.	Automatically generated values from constrained strategies.
Primary oracle style	Concrete expected values and database assertions.	General arithmetic, relational, and state invariants.
Boundary treatment	Explicit values below, at, and above thresholds.	Generated ranges that include valid and invalid regions.
Breadth	Broader explicit coverage of account and transaction helpers.	Narrower property catalogue focused on core financial invariants.
Failure diagnosis	Directly tied to the chosen example.	Hypothesis shrinking can produce a minimal counterexample.
Principal risk	Important cases may be omitted by the tester.	Important behaviours may be omitted from the property catalogue.

The suites were therefore different not only in the number of test functions, but also in how they represented the specification. The manual suite emphasized explicit behavioural breadth, while the property-based suite emphasized generalization over generated inputs.

7 Execution Results and Coverage Comparison

7.1 Test Execution Results

Both test suites executed successfully against the pinned subject-system revision.

The manually designed suite contained 29 pytest test functions. All 29 tests passed.

The property-based suite contained 16 Hypothesis test functions. All 16 tests passed over their generated examples.

Table 9: Test-suite execution summary

Test suite	Test functions	Result	Testing approach
Manual EP/BVA suite	29	29 passed	Fixed examples selected using equivalence partitioning and boundary value analysis.
Property-based suite	16	16 passed	Hypothesis properties executed over automatically generated inputs.

The number of test functions should not be interpreted as the number of individual input executions. Each Hypothesis test function may execute many generated examples during a single pytest run. Therefore, the comparison focuses on behavioural coverage and mutation adequacy rather than raw test function count alone.

7.2 Overall Structural Coverage

The manual suite achieved 84% total coverage across the selected production modules. The property-based suite achieved 77%.

Table 10: Overall structural coverage comparison

Test suite	Statement and branch scope	Coverage	Difference
Manual EP/BVA suite	Selected financial modules	84%	7 percentage points
Property-based suite	Selected financial modules	77%	

7.3 Per-Module Coverage

Table 11 presents the coverage achieved by each suite for the selected production modules.

Table 11: Per-module structural coverage comparison

Production module	Manual suite	Property suite	Difference
accounts/models.py	96%	79%	17 points
transactions/forms.py	75%	75%	0 points
transactions/models.py	100%	92%	8 points
transactions/views.py	79%	75%	4 points
Overall	84%	77%	7 points

The largest difference occurred in `accounts/models.py`, where the manual suite achieved 96% coverage compared with 79% for the property-based suite. This 17-point difference is consistent with the broader set of account-model helper behaviours represented in the manual test catalogue.

Both suites achieved 75% coverage for `transactions/forms.py`. This indicates that they reached a similar proportion of the form implementation, although equal structural coverage does not necessarily imply equal mutation effectiveness.

The manual suite achieved complete coverage of `transactions/models.py`, while the property-based suite achieved 92%. The difference suggests that at least one model behaviour or branch was not directly exercised by the property catalogue.

Coverage of `transactions/views.py` was relatively close, with 79% for the manual suite and 75% for the property-based suite.

7.4 Coverage Interpretation

The coverage results show that the manually designed suite exercised a broader portion of the selected subsystem. However, coverage does not establish that the executed behaviour was checked strongly enough to detect faults.

For example, two suites may execute the same statement while using different assertions. One suite may verify the exact resulting balance, while another may verify only that the operation completed successfully. Both tests may contribute equally to structural coverage even though their fault-detection strength differs.

For this reason, coverage was treated as a supporting metric rather than the primary adequacy criterion.

7.5 Coverage Evidence Artifacts

The complete coverage outputs were retained in the repository:

```
results/coverage/manual_coverage.txt
results/coverage/manual_coverage.json
results/coverage/property_coverage.txt
results/coverage/property_coverage.json
```


The text reports preserve the command-line summaries, while the JSON reports support programmatic comparison and chart generation.

7.6 Coverage Comparison

Figure 1 presents the module-level structural coverage achieved by the two test suites.

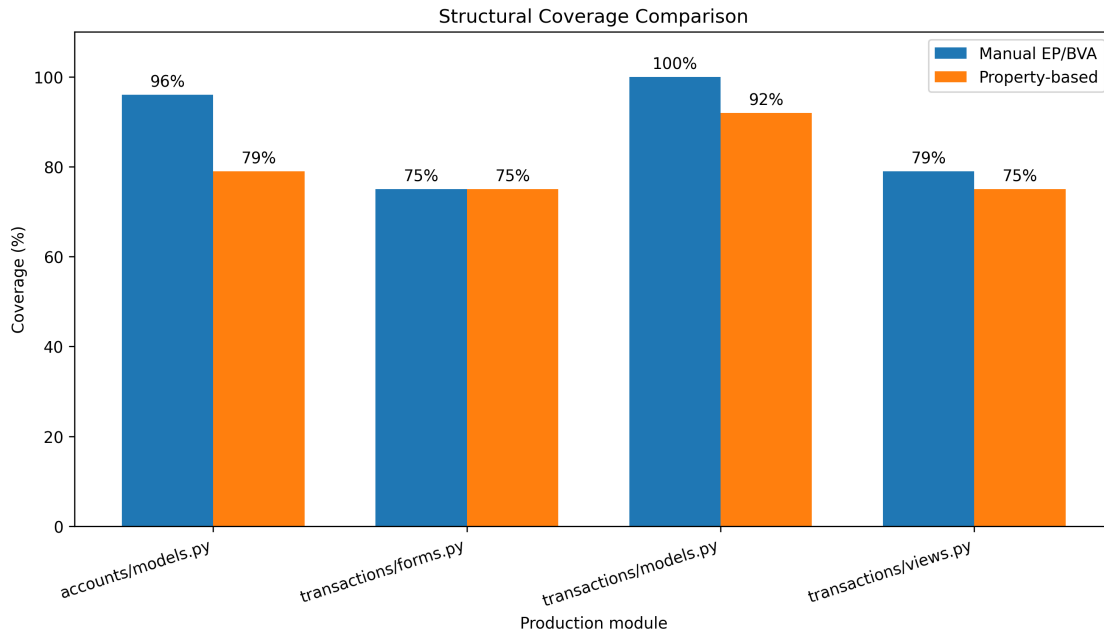


Figure 1: Structural coverage achieved by the manual and property-based test suites for each selected production module.

As shown in Figure 1, the manual suite achieved equal or higher coverage for every selected module. The largest difference occurred in `accounts/models.py`, where the manual suite achieved 96% coverage compared with 79% for the property-based suite.

8 Mutation Testing Results

8.1 Generated Mutant Population

Mutation testing was executed separately for the manual and property-based test suites using the same production-code scope and the same mutation configuration.

Each run generated 96 mutants from the same pinned source revision and identical mutation configuration. The matching counts, target files, and configuration provide evidence that the mutant population remained consistent between runs.

Table 12: Mutation-testing outcome summary

Test suite	Generated	Killed	Survived	No tests	Timeout
Manual EP/BVA suite	96	53	17	26	0
Property-based suite	96	42	15	39	0

The manual suite killed 53 mutants, while the property-based suite killed 42. The property-based suite also produced a larger number of no-test outcomes, indicating that more mutated behaviours were outside its execution reach.

No timeout mutants occurred in either run.

8.2 Raw Mutation Outcomes

The raw outcome percentages were calculated relative to the complete population of 96 mutants.

For the manual suite:

$$\text{Killed proportion} = \frac{53}{96} \times 100 = 55.21\% \quad (5)$$

$$\text{Survived proportion} = \frac{17}{96} \times 100 = 17.71\% \quad (6)$$

$$\text{No-test proportion} = \frac{26}{96} \times 100 = 27.08\% \quad (7)$$

For the property-based suite:

$$\text{Killed proportion} = \frac{42}{96} \times 100 = 43.75\% \quad (8)$$

$$\text{Survived proportion} = \frac{15}{96} \times 100 = 15.63\% \quad (9)$$

$$\text{No-test proportion} = \frac{39}{96} \times 100 = 40.63\% \quad (10)$$

These raw percentages are useful for describing the complete tool output. However, they are not the final controlled mutation scores because the generated population included 26 mutants from a feature that was declared outside the experiment scope.

8.3 Controlled Relevant Mutant Population

The 26 mutants associated with `TransactionDateRangeForm.clean_daterange` were outside the selected financial transaction scope.

These mutants were classified as no tests in both experimental runs. They were therefore excluded consistently from the controlled comparison.

The relevant population was:

$$M_{\text{relevant}} = 96 - 26 = 70 \quad (11)$$

For the manual suite, all 70 relevant mutants were reached:

$$53 + 17 = 70 \quad (12)$$

For the property-based suite, 57 relevant mutants were reached and 13 relevant mutants were classified as no tests:

$$42 + 15 + 13 = 70 \quad (13)$$

8.4 Controlled Mutation Scores

The controlled mutation score used the same denominator of 70 relevant mutants for both suites.

For the manual suite:

$$MS_{\text{manual}} = \frac{53}{70} \times 100 = 75.71\% \quad (14)$$

For the property-based suite:

$$MS_{\text{property}} = \frac{42}{70} \times 100 = 60.00\% \quad (15)$$

The difference was:

$$75.71\% - 60.00\% = 15.71 \text{ percentage points} \quad (16)$$

Table 13: Controlled mutation-score comparison

Test suite	Killed relevant mutants	Relevant population	Controlled score
Manual EP/BVA suite	53	70	75.71%
Property-based suite	42	70	60.00%

Under this controlled interpretation, the manual suite achieved the higher mutation score.

8.5 Reached-Mutant Scores

The controlled score combines two factors:

- whether the suite reaches the mutated behaviour; and
- whether its oracle detects the behavioural change.

To examine oracle effectiveness within reached code, a second score was calculated using only killed and surviving mutants.

For the manual suite:

$$MS_{\text{manual, reached}} = \frac{53}{53 + 17} \times 100 = 75.71\% \quad (17)$$

For the property-based suite:

$$MS_{\text{property, reached}} = \frac{42}{42 + 15} \times 100 = 73.68\% \quad (18)$$

Table 14: Reached-mutant mutation-score comparison

Test suite	Killed	Survived	Reached-mutant score
Manual EP/BVA suite	53	17	75.71%
Property-based suite	42	15	73.68%

The reached-mutant scores differed by only 2.03 percentage points. This is substantially smaller than the 15.71-point difference observed using the controlled relevant population.

This result suggests that the main difference between the suites was behavioural reach. Once mutated code was reached, their fault-detection effectiveness was relatively similar.

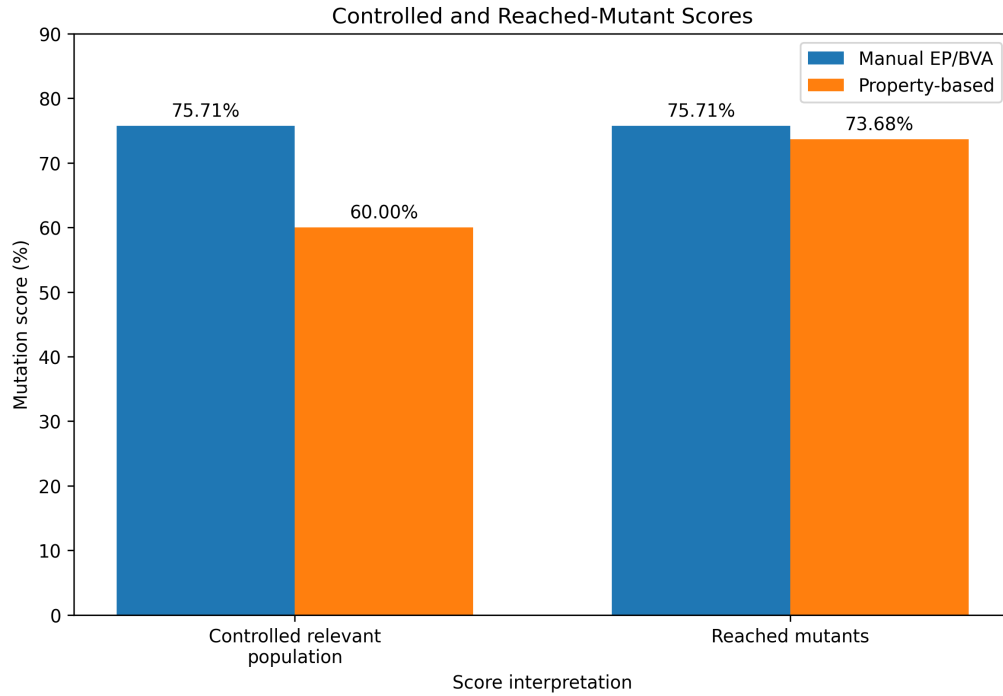


Figure 2: Comparison of controlled relevant-population and reached-mutant mutation scores.

Figure 2 shows that the controlled scores differed by 15.71 percentage points, while the reached-mutant scores differed by only 2.03 percentage points. This supports the conclusion that behavioural reach was the principal source of the overall difference.

8.6 Interpretation of No-Test Outcomes

The manual suite produced 26 no-test mutants, all of which belonged to the known out-of-scope date-range form.

The property-based suite produced 39 no-test mutants. After removing the same 26 out-of-scope mutants, 13 relevant mutants remained untested.

These additional no-test outcomes are consistent with the narrower property catalogue and the lower structural coverage observed in `accounts/models.py`, `transactions/models.py`, and `transactions/views.py`.

The no-test classification should not be interpreted as an assertion failure. It indicates that the selected suite did not execute the mutated code.

8.7 Mutation Result Artifacts

The complete mutation outputs were preserved under:

```
results/mutation/manual/  
results/mutation/property_based/
```

Each directory contains:

- the complete mutant result list;
- the summary output;
- surviving-mutant identifiers;
- setup information; and
- exported survivor details.

A consolidated comparison was stored in:

```
results/mutation/comparison_summary.txt
```

8.8 Mutation Outcome Comparison

Figure 3 compares the raw mutation outcomes for the manual and property-based test suites across the consistently generated population of 96 mutants.

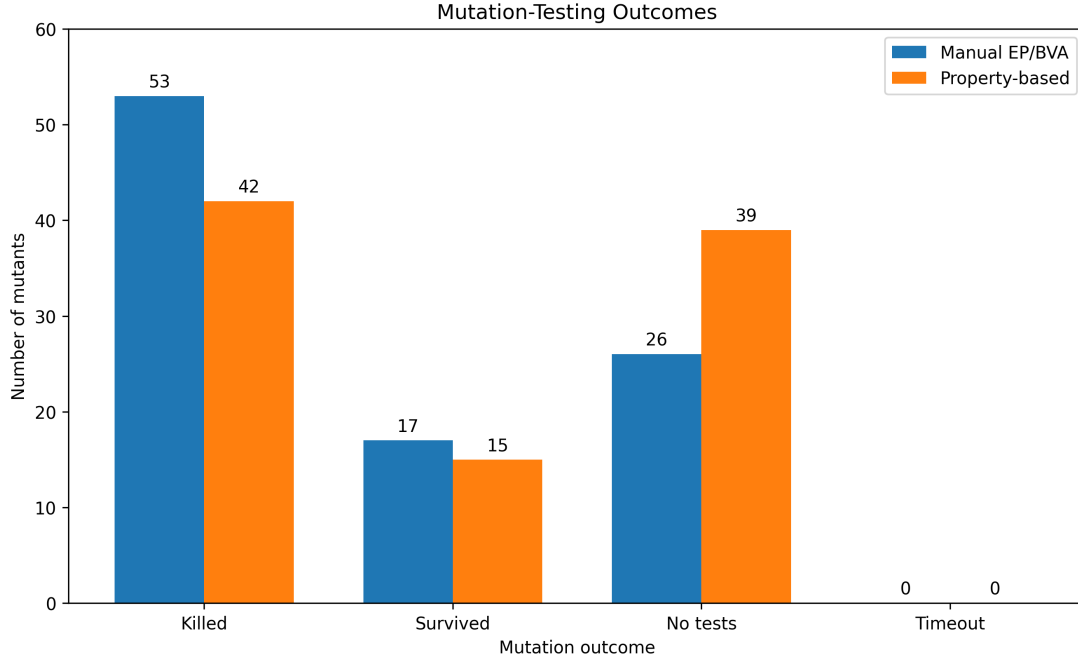


Figure 3: Mutation-testing outcomes for the manual and property-based test suites.

As shown in Figure 3, the manual suite killed more mutants and produced fewer no-test outcomes. The manual suite killed 53 mutants and produced 26 no-test outcomes, whereas the property-based suite killed 42 mutants and produced 39 no-test outcomes. This result supports the interpretation that the manual suite reached a broader portion of the selected behavioural scope.

9 Surviving-Mutant Analysis

9.1 Purpose of Survivor Analysis

Mutation score alone does not explain why mutants survive. A surviving mutant may indicate a genuine weakness in the test suite, an equivalent change, an underspecified requirement, or behaviour outside the study scope.

For this reason, the surviving mutants were inspected individually. The manual suite produced 17 unique survivors, and each was classified according to its semantic effect and relationship to the reconstructed requirements.

9.2 Survivor Classification

The 17 manual-suite survivors were classified as follows:

Table 15: Classification of unique surviving mutants

Classification	Count	Interpretation
Genuine test weakness	7	The mutant changed financially relevant behaviour, but the existing tests did not detect the change.
Equivalent or likely equivalent	2	The source change did not appear to alter observable behaviour for the valid input domain considered in the study.
Underspecified diagnostic text	7	The mutant altered validation-message wording or formatting, while the financial acceptance or rejection behaviour remained unchanged.
Outside core scope	1	The mutant affected behaviour that was not part of the principal financial verification objective.
Total	17	

The classification shows that fewer than half of the survivors represented clear financial test weaknesses. Several survived because the study intentionally did not specify exact diagnostic text.

9.3 Genuine Test Weaknesses

Seven survivors were classified as genuine weaknesses. These mutants were concentrated in four areas:

- interest arithmetic;
- monetary rounding;
- interest-calculation scheduling; and
- positional form initialization.

9.3.1 Interest Arithmetic

Some mutants changed arithmetic used in the interest calculation. These survivors showed that the existing tests did not distinguish all relevant operator or constant substitutions.

A general interest oracle was present, but certain generated or fixed examples did not make the mutated and original expressions sufficiently different. This demonstrates that a mathematically meaningful property can still be too weak when the selected input domain does not expose the change.

A stronger test design would include independently calculated expected values for several combinations of principal, rate, and calculation frequency.

9.3.2 Monetary Rounding

Mutants affecting rounding behaviour also survived. Financial software should handle currency precision explicitly, since small rounding differences may accumulate or produce inconsistent recorded balances.

The existing tests checked two-decimal output in some cases, but they did not include enough values near rounding boundaries.

Additional examples should include values whose third decimal digit is:

- below five;
- exactly five; and
- above five.

This would increase sensitivity to mutations affecting rounding precision or rounding position.

9.3.3 Interest-Calculation Scheduling

The manual suite included explicit examples for interest-calculation month scheduling. However, the selected assertions did not distinguish every mutated variation of the scheduling logic. This demonstrates that executing a helper method and checking selected outputs may still leave alternative mutated schedules observationally equivalent for the chosen examples.

The property-based suite did not directly represent this behaviour in its original property catalogue. This omission also contributed to its reduced reach within `accounts/models.py`.

A stronger oracle would verify:

- the exact number of calculated months;

- the first expected month;
- the interval between successive months;
- the absence of duplicates; and
- valid month values between 1 and 12.

9.3.4 Positional Form Initialization

One surviving mutant affected positional initialization behaviour in a transaction form. The original tests primarily constructed the form using keyword arguments.

As a result, a mutation affecting positional argument handling was not detected. A dedicated test should instantiate the form using the supported positional calling pattern and verify that account and transaction-type information are assigned correctly.

9.4 Equivalent or Likely Equivalent Mutants

Two survivors were classified as equivalent or likely equivalent.

Equivalent mutants are syntactically different from the original program but produce the same observable behaviour for all relevant inputs. Such mutants cannot be killed by adding a valid test unless the specification is changed.

Because proving semantic equivalence is generally difficult, these mutants were classified conservatively as likely equivalent based on source inspection and repeated execution over the relevant input domain.

They were retained in the raw tool output but distinguished from genuine test weaknesses during interpretation.

9.5 Underspecified Diagnostic-Text Mutants

Seven survivors altered validation-message text without changing the financial decision.

For example, a mutant may change capitalization, punctuation, or wording while the same invalid transaction is still rejected.

The reconstructed requirements specified whether an input must be accepted or rejected,

but they did not define exact diagnostic strings. Therefore, tests that assert only validation failure are consistent with the specification.

These survivors do not demonstrate incorrect financial behaviour. Instead, they expose underspecified presentation-level behaviour.

Requiring exact message text would increase the mutation score, but it would also make the suite more brittle and more tightly coupled to user-interface wording.

9.6 Outside-Scope Survivor

One survivor affected behaviour outside the principal financial verification scope.

This mutant was documented but was not treated as evidence of weakness in the core deposit, withdrawal, balance, transaction, or interest requirements.

9.7 Manual and Property Survivor Relationship

The manual suite produced 17 survivors, while the property-based suite produced 15. These counts alone should not be used to conclude that the property suite was stronger.

The property suite reached only 57 of the 70 relevant mutants. Therefore, some mutants that survived under the manual suite may have been classified as no tests under the property suite rather than as killed.

The reached-mutant scores provide a fairer comparison of oracle effectiveness:

$$MS_{\text{manual, reached}} = 75.71\%$$

$$MS_{\text{property, reached}} = 73.68\%$$

These close scores indicate that the principal difference was reach rather than a large difference in detection strength within executed behaviour.

9.8 Lessons from Survivor Analysis

The survivor analysis produced four important lessons.

First, broad code execution does not guarantee strong test oracles. Some financial arithmetic was reached, but the existing assertions did not distinguish the original implementation from the mutated version.

Second, property-based testing depends on the completeness of the property catalogue. Generating many inputs cannot compensate for an omitted behaviour or helper method.

Third, exact validation-message assertions should be introduced only when the message wording is explicitly part of the specification. Otherwise, such tests may increase brittleness without improving verification of financial correctness.

Fourth, mutation results are more informative when surviving mutants are examined semantically rather than reported only as a percentage. Survivor classification helps distinguish genuine test weaknesses from equivalent, underspecified, or out-of-scope behaviour.

9.9 Survivor Analysis Artifact

The complete survivor analysis was preserved in:

```
docs/surviving_mutant_analysis.md
```

Detailed source-level mutant information was also exported as follows:

```
results/mutation/survivor_details/
```

These artifacts record the mutant location, source change, classification, and recommended test improvement.

10 Discussion

10.1 Answer to the Primary Research Question

The primary research question asked how the mutation adequacy of a manually designed equivalence-partitioning and boundary-value suite compared with that of a Hypothesis

property-based suite for the selected financial subsystem.

Using the controlled population of 70 relevant mutants, the manual suite achieved a mutation score of 75.71%, while the property-based suite achieved 60.00%. The manual suite therefore achieved the higher controlled mutation score by 15.71 percentage points.

This result does not mean that manually designed testing is universally more effective than property-based testing. In this experiment, the manual suite represented a broader range of account, transaction, and helper behaviours. The property-based suite focused more narrowly on the principal financial invariants.

When only reached mutants were considered, the difference was much smaller. The manual suite achieved 75.71%, while the property-based suite achieved 73.68%. This 2.03-point difference indicates that the two suites had similar fault-detection effectiveness within the code they actually exercised.

The main experimental difference was therefore behavioural reach rather than a large difference in oracle strength.

10.2 RQ1: Which Suite Achieved Higher Structural Coverage?

The manual suite achieved higher overall structural coverage.

- Manual suite: 84%.
- Property-based suite: 77%.
- Difference: 7 percentage points.

The largest module-level difference occurred in `accounts/models.py`, where the manual suite achieved 96% coverage and the property-based suite achieved 79%.

The manual suite also achieved higher coverage in `transactions/models.py` and `transactions/views.py`. Both suites achieved 75% in `transactions/forms.py`.

These results support the conclusion that the manual suite represented a broader portion of the selected subsystem.

10.3 RQ2: Which Suite Reached a Broader Range of Financial Behaviour?

The mutation no-test outcomes provide direct evidence about behavioural reach.

After excluding the 26 common out-of-scope mutants, the manual suite reached all 70 relevant mutants. The property-based suite reached 57 and did not reach 13 relevant mutants.

The property-based suite’s lower reach was mainly associated with behaviours that were absent from its property catalogue, including selected account-model and helper functionality.

This result illustrates an important limitation of property-based testing: automatic generation can explore many values only within the behaviours that have been encoded as properties. It does not automatically discover missing requirements or omitted helper methods.

10.4 RQ3: Which Categories of Mutants Survived?

The surviving mutants fell into four principal categories:

1. genuine financial test weaknesses;
2. equivalent or likely equivalent changes;
3. underspecified diagnostic-message changes; and
4. behaviour outside the principal scope.

The genuine weaknesses were concentrated in interest arithmetic, monetary rounding, interest-calculation scheduling, and positional form initialization.

The diagnostic-message survivors show that mutation testing can reveal underspecified requirements as well as weak tests. The tests correctly checked financial rejection behaviour, but they did not require exact wording because the specification did not define it.

10.5 RQ4: What Relationship Was Observed Between Coverage and Mutation Adequacy?

The manual suite achieved both higher structural coverage and a higher controlled mutation score.

However, the relationship was not one-to-one. The overall coverage difference was 7 percentage points, while the controlled mutation-score difference was 15.71 percentage points.

This occurred because mutation score reflects both execution and detection. Coverage measures whether code is executed, while mutation testing also checks whether assertions distinguish the original behaviour from a modified version.

The reached-mutant scores were close even though the overall coverage values differed. This further supports the interpretation that the manual suite’s main advantage was broader reach.

10.6 Manual EP/BVA Strengths

The manual suite had several strengths.

First, the explicit requirements-to-test traceability encouraged broad representation of the subsystem.

Second, equivalence partitioning ensured that valid and invalid classes were considered systematically.

Third, boundary value analysis directly targeted financial thresholds such as minimum deposit, minimum withdrawal, maximum withdrawal, and available balance.

Fourth, fixed examples supported precise arithmetic and persistence assertions.

These strengths contributed to the suite’s broader structural and mutation reach.

10.7 Manual EP/BVA Limitations

The manual approach depended strongly on tester judgement.

A partition, boundary, or helper behaviour could be missed if it was not identified during design. Fixed examples may also fail to expose faults that occur only for unusual numeric combinations.

Manual tests can therefore provide strong traceability and breadth, but their input diversity remains limited unless many examples are added.

10.8 Property-Based Testing Strengths

The property-based suite expressed financial behaviour in a reusable and general form.

Generated values allowed the same invariant to be evaluated across many inputs. Relational properties such as exact balance change and monotonicity were natural to express using Hypothesis.

Shrinking also improved failure diagnosis by reducing complex failing examples to simpler counterexamples.

Within reached code, the property-based suite achieved a reached-mutant score close to that of the manual suite. This indicates that its financial properties were generally effective when they exercised the relevant behaviour.

10.9 Property-Based Testing Limitations

The main limitation was catalogue completeness.

The property suite did not directly represent every selected account and transaction helper. As a result, 13 relevant mutants were not reached.

Some properties may also be too general to detect particular arithmetic or rounding mutations. A property such as non-negativity can pass even when the computed value is financially incorrect.

Property-based testing therefore requires both strong invariants and broad behavioural traceability.

10.10 Complementary Use of Both Approaches

The results support treating the two approaches as complementary rather than as substitutes.

Manual EP/BVA testing is effective for:

- explicit requirement traceability;
- known input partitions;
- threshold-focused validation;

- helper-method coverage; and
- precise expected-value examples.

Property-based testing is effective for:

- exploring many values within defined domains;
- checking general invariants;
- finding unexpected numeric combinations;
- testing relational properties; and
- minimizing counterexamples through shrinking.

A stronger combined strategy would use EP/BVA to ensure behavioural breadth and Hypothesis to deepen exploration within each identified requirement.

10.11 Implications for Financial Software Testing

Financial software requires more than successful execution. Tests should verify exact monetary effects, state preservation, persistence consistency, and rounding behaviour.

The survivor analysis showed that weak arithmetic or precision checks can allow financially significant mutations to survive. It also showed that broadly stated properties may not be sufficient unless they are paired with independent expected-value calculations.

For financial systems, a robust suite should therefore include:

- explicit boundary cases;
- generated valid and invalid amounts;
- exact before-and-after balance assertions;
- independent arithmetic oracles;
- rounding-boundary examples;
- persistence checks; and
- mutation analysis to evaluate oracle strength.

10.12 Overall Interpretation

The study does not establish that one test-design technique is always superior.

For this subject system and experimental scope, the manual suite achieved better overall adequacy because it covered a broader set of behaviours. The property-based suite was nearly as effective within reached code, but its property catalogue was narrower.

The most defensible conclusion is therefore:

Manual EP/BVA testing provided broader behavioural coverage in this case study, while property-based testing provided comparable fault-detection strength within the behaviours it exercised. The strongest practical strategy is to combine explicit requirement-based breadth with generated property-based depth.

11 Threats to Validity

11.1 Internal Validity

Internal validity concerns whether the observed differences were caused by the test-design approaches rather than by uncontrolled experimental factors.

Several controls were applied. Both suites were executed against the same pinned source revision, the same selected production modules, the same Python environment, the same database setup, and the same mutation tool. Both mutation runs generated 96 mutants, providing evidence that the mutant population was stable.

However, some internal-validity threats remain.

First, the two suites were not equal in behavioural breadth. The manual suite explicitly covered more helper methods, while the property-based suite focused on a narrower property catalogue. Therefore, the experiment compares the implemented suites as designed, not two perfectly size-matched or scope-matched artifacts.

Second, manual judgement influenced both requirement extraction and survivor classification. Another tester might identify different equivalence classes, write different properties, or classify ambiguous mutants differently.

Third, the tests were created with knowledge of the implementation structure. Although the

manual suite was based on EP/BVA and the property suite was based on invariants, source inspection was required to reconstruct requirements and configure the experiment. This may have influenced test selection.

11.2 Construct Validity

Construct validity concerns whether the selected measurements accurately represent test adequacy.

Structural coverage was measured using statement and branch coverage. These metrics indicate execution but do not directly measure oracle strength.

Mutation score was therefore used as the principal adequacy measure. However, mutation testing is only a proxy for real fault detection. The generated mutants may not fully represent realistic financial defects.

Equivalent and underspecified mutants also affect interpretation. A surviving mutant may not indicate a test weakness when the modified program is semantically equivalent or when the changed behaviour is not part of the requirements.

To reduce this threat, raw mutation outcomes, controlled mutation scores, reached-mutant scores, and semantic survivor classifications were reported separately.

11.3 External Validity

External validity concerns whether the findings generalize beyond this case study.

The experiment used one open-source Django banking application and one selected financial subsystem. The findings may not generalize directly to other applications, programming languages, frameworks, domains, or mutation tools.

The subject system also lacked a direct inter-account transfer feature. Therefore, traditional transfer-conservation properties were replaced with deposit, withdrawal, and state-preservation invariants.

The relative performance of manual and property-based testing may differ in a system with more complex calculations, concurrency, distributed transactions, or a larger existing test suite.

Accordingly, the conclusions are limited to this subject system, source revision, toolchain,

and declared scope.

11.4 Conclusion Validity

Conclusion validity concerns whether the numerical evidence supports the reported interpretation.

The experiment used one execution configuration and one pair of test suites. No statistical hypothesis test was performed because the primary data consisted of deterministic coverage and mutation outcomes for a single pinned revision.

The controlled mutation-score difference was 15.71 percentage points, while the reached-mutant difference was only 2.03 percentage points. This distinction was important for avoiding an overly broad conclusion that the manual suite had much stronger assertions.

The results support the narrower conclusion that the manual suite had broader behavioural reach, while the two suites had similar effectiveness within reached code.

11.5 Reliability and Reproducibility

Reliability concerns whether another investigator could repeat the experiment and obtain comparable results.

The project improves repeatability by preserving:

- the pinned upstream revision;
- dependency versions;
- separate test-suite directories;
- coverage commands and reports;
- mutation setup scripts;
- raw mutation outputs;
- exported survivor details; and
- the survivor-classification document.

Despite these controls, exact reproduction may still depend on operating-system behaviour, dependency availability, database configuration, or future changes in external tools.

11.6 Researcher Bias

The same researcher designed the tests, executed the experiment, interpreted the outputs, and classified surviving mutants. This creates a risk of confirmation bias.

To reduce this risk, classifications were based on explicit criteria:

- whether the mutant changed observable financial behaviour;
- whether the behaviour was included in the declared requirements;
- whether the suite reached the mutated code;
- whether the existing oracle could observe the change; and
- whether the mutant appeared equivalent for the relevant input domain.

The detailed mutant artifacts were retained so that the classifications can be reviewed independently.

11.7 Mitigation Summary

The principal mitigation measures were:

- identical source and mutation scope for both suites;
- a stable generated mutant population;
- separate execution of each suite;
- explicit requirements and invariants;
- preservation of raw artifacts;
- separate reporting of raw, controlled, and reached-mutant results;
- semantic survivor inspection; and
- cautious limitation of the final conclusions.

These measures do not eliminate all threats, but they make the experimental assumptions and limitations visible.

12 Reproduction Instructions and Practical Artifact

12.1 Artifact Overview

The practical artifact contains the subject system, both test suites, requirements and invariant documents, coverage reports, mutation outputs, surviving-mutant details, and automation scripts.

The repository is available at:

<https://github.com/hadiChowdhury/Svv-banking-verification-study>

The principal repository structure is:

```
Svv-banking-verification-study/  
|-- subject_system/  
|   '-- banking_system/  
|-- tests/  
|   |-- manual/  
|   '-- property_based/  
|-- docs/  
|-- results/  
|   |-- coverage/  
|   '-- mutation/  
|-- scripts/  
|-- requirements.txt  
|-- pytest.ini  
'-- README.md
```

12.2 Prerequisites

The experiment requires:

- Python 3.10;
- Git;
- a command-line terminal; and
- the dependencies listed in `requirements.txt`.

The experiment was originally executed using Python 3.10.20. Using the same major and minor Python version is recommended because the subject system uses an older Django release.

12.3 Repository Setup

Clone the repository and enter the project directory:

Listing 1: Cloning the project repository

```
1 git clone https://github.com/hadiChowdhury/Svv-banking-verification-study.git
2 cd Svv-banking-verification-study
```

Create a virtual environment:

Listing 2: Creating a Python virtual environment

```
1 python3.10 -m venv .venv
```

Activate it on macOS or Linux:

Listing 3: Activating the environment on macOS or Linux

```
1 source .venv/bin/activate
```

On Windows PowerShell:

Listing 4: Activating the environment on Windows

```
1 .venv\Scripts\Activate.ps1
```

Install the dependencies:

Listing 5: Installing project dependencies

```
1 python -m pip install --upgrade pip
2 pip install -r requirements.txt
```

12.4 Executing the Manual Suite

The manual EP/BVA suite was executed independently using the following verified command:

Listing 6: Executing the manual EP/BVA suite

```
1 pytest -c mutmut_pytest.ini tests/manual -q
```

The observed result was:

29 passed

12.5 Executing the Property-Based Suite

The property-based suite was executed independently using the following verified command:

Listing 7: Executing the property-based suite

```
1 pytest -c mutmut_pytest.ini tests/property_based -q
```

The observed result was:

16 passed

A passing Hypothesis test function may execute many generated examples. Therefore, the reported 16 test functions should not be interpreted as only 16 individual input executions.

12.6 Collecting Manual-Suite Coverage

Manual-suite statement and branch coverage were collected using the following verified command:

Listing 8: Collecting manual-suite coverage

```
1 pytest -c mutmut_pytest.ini tests/manual \  
2 --cov=accounts.models \  
3 --cov=transactions.forms \  
4 --cov=transactions.models \  
5 --cov=transactions.views \  
6 --cov-branch \  
7 --cov-report=term-missing
```


The run completed with 29 passing tests and produced the following coverage results:

- 96% for `accounts/models.py`;
- 75% for `transactions/forms.py`;
- 100% for `transactions/models.py`;
- 79% for `transactions/views.py`; and
- 84% overall.

The preserved outputs are available in:

```
results/coverage/manual_coverage.txt  
results/coverage/manual_coverage.json
```

12.7 Collecting Property-Suite Coverage

Property-based statement and branch coverage were collected using the following verified command:

Listing 9: Collecting property-based coverage

```
1 pytest -c mutmut_pytest.ini tests/property_based \  
2   --cov=accounts.models \  
3   --cov=transactions.forms \  
4   --cov=transactions.models \  
5   --cov=transactions.views \  
6   --cov-branch \  
7   --cov-report=term-missing
```

The run completed with 16 passing tests and produced the following coverage results:

- 79% for `accounts/models.py`;
- 75% for `transactions/forms.py`;
- 92% for `transactions/models.py`;
- 75% for `transactions/views.py`; and

- 77% overall.

The preserved outputs are available in:

```
results/coverage/property_coverage.txt  
results/coverage/property_coverage.json
```

12.8 Preparing the Mutation Workspace

The repository includes a script for preparing a clean mutation-testing workspace:

Listing 10: Preparing the mutation workspace

```
1 bash scripts/prepare_mutation_workspace.sh
```

The mutation configuration must be checked before each run so that the correct test suite is selected.

The manual mutation run must invoke only the manual suite. The property-based mutation run must invoke only the property-based suite.

12.9 Executing the Manual Mutation Run

The saved manual mutation configuration was restored before creating a clean mutation workspace:

Listing 11: Executing the manual mutation run

```
1 cp results/mutation/manual/setup.cfg setup.cfg  
2 rm -rf mutants  
3 mutmut run
```

The observed manual mutation result was:

```
Generated: 96  
Killed:    53  
Survived:  17  
No tests:  26  
Timeout:   0
```

12.10 Executing the Property-Based Mutation Run

The property-based configuration was restored and the previous mutation workspace was removed:

Listing 12: Executing the property-based mutation run

```
1 cp results/mutation/property_based/setup.cfg setup.cfg
2 rm -rf mutants
3 mutmut run
```

The observed property-based mutation result was:

```
Generated: 96
Killed:    42
Survived:  15
No tests:  39
Timeout:   0
```

12.11 Inspecting Mutation Results

After either run, surviving-mutant results can be displayed using:

Listing 13: Displaying mutation results

```
1 mutmut results
```

12.12 Exporting Surviving Mutants

The repository contains a script for exporting surviving-mutant details:

Listing 14: Exporting surviving-mutant details

```
1 bash scripts/export_surviving_mutants.sh
```

The exported artifacts include mutant identifiers and source-level changes used during semantic classification.

12.13 Generated Result Artifacts

The experiment generated the following practical artifacts:

Table 16: Principal reproducibility artifacts

Artifact	Purpose
docs/requirements_specification.md	Documents the reconstructed financial requirements.
docs/invariants.md	Records the financial properties used as test oracles.
docs/results_analysis.md	Explains the coverage and mutation comparisons.
docs/surviving_mutant_analysis.md	Classifies survivors and recommends test improvements.
results/coverage/	Contains manual and property-based coverage reports.
results/mutation/manual/	Contains raw manual-suite mutation outputs.
results/mutation/property_based/	Contains raw property-suite mutation outputs.
results/mutation/comparison_summary.txt	Provides the consolidated mutation comparison.
scripts/	Contains scripts used to prepare and export mutation results.

12.14 Observed Experimental Summary

The observed experimental results are summarized in Table ?? . A reproduction using the pinned environment should yield the same numerical outcomes.

Table 17: Expected reproduction results

Measurement	Manual suite	Property suite
Passing test functions	29	16
Overall coverage	84%	77%
Generated mutants	96	96
Killed mutants	53	42
Surviving mutants	17	15
No-test mutants	26	39
Controlled mutation score	75.71%	60.00%
Reached-mutant score	75.71%	73.68%

12.15 Reproduction Limitations

Minor differences may occur if the experiment is executed using another Python version, operating system, dependency release, or mutation-tool version.

For the closest reproduction, the pinned dependency versions and upstream revision should

be preserved. The mutation workspace should also be cleaned between the manual and property-based runs.

13 Recommendations and Future Work

13.1 Recommended Test Improvements

The mutation analysis identified several concrete opportunities to strengthen both test suites.

First, interest calculations should be checked using independently derived expected values across a wider range of principals, rates, and calculation frequencies. This would improve sensitivity to arithmetic-operator and constant mutations.

Second, monetary rounding should be tested explicitly near decimal boundaries. Useful cases include values whose third decimal digit is below five, exactly five, and above five.

Third, the property catalogue should be expanded to include interest-calculation month scheduling. A suitable property should verify the number of scheduled months, the interval between months, valid month values, and the absence of duplicates.

Fourth, dedicated tests should exercise supported positional form initialization in addition to keyword-based construction.

Fifth, requirement traceability should be maintained for both suites. Every selected financial requirement should map to at least one manual test and at least one property or an explicitly documented justification for omission.

13.2 Recommended Combined Strategy

The results suggest that the strongest practical approach is a hybrid strategy.

Equivalence partitioning and boundary value analysis should first be used to identify the behavioural scope, invalid regions, valid regions, and important thresholds.

Property-based testing should then be applied within each identified behaviour to explore a larger range of values and relational cases.

A recommended process is:

1. extract and document financial requirements;
2. identify equivalence partitions and boundaries;
3. create at least one fixed example for each partition and boundary;
4. define one or more general properties for each relevant behaviour;
5. generate diverse monetary and account-state inputs;
6. measure statement and branch coverage;
7. evaluate the suite using mutation testing;
8. inspect surviving and no-test mutants; and
9. refine the requirements and tests iteratively.

This process combines the breadth of requirements-based manual design with the input diversity of property-based testing.

13.3 Future Work

Several extensions could strengthen the study.

First, the experiment could be repeated on additional financial applications. Using multiple subject systems would improve external validity and show whether the observed reach difference is specific to this Django project.

Second, future work could construct scope-matched suites. Every selected behaviour could be represented in both the manual and property-based catalogues, allowing a more direct comparison of input-generation style and oracle strength.

Third, the number of generated Hypothesis examples could be varied systematically. This would help determine whether mutation score changes as the generated input budget increases.

Fourth, additional mutation tools or operator sets could be used to assess whether the findings depend on the specific behaviour of `mutmut`.

Fifth, mutation analysis could be extended to realistic seeded faults or known historical defects. This would provide evidence about whether mutation score corresponds to detection of realistic financial errors.

Sixth, the study could include performance, concurrency, and transactional integrity properties. These concerns are particularly important in production banking systems but were outside the present scope.

Seventh, the surviving diagnostic-message mutants could be revisited after defining explicit usability or interface requirements. Exact message tests would be justified only when the wording itself becomes part of the specification.

14 Conclusion

This project compared two software-testing approaches applied to the financial transaction subsystem of an open-source Django banking application. The first suite used manually designed pytest tests based on equivalence partitioning and boundary value analysis. The second suite used Hypothesis to evaluate general financial properties over generated inputs.

Both suites passed against the pinned subject-system revision. The manual suite contained 29 test functions and achieved 84% structural coverage. The property-based suite contained 16 property test functions and achieved 77% coverage.

Mutation testing generated the same population of 96 mutants for both runs. Twenty-six common mutants belonged to an out-of-scope date-range form and were excluded consistently, leaving a controlled population of 70 relevant mutants.

The manual suite killed 53 relevant mutants and achieved a controlled mutation score of 75.71%. The property-based suite killed 42 relevant mutants and achieved 60.00%. The manual suite therefore achieved the higher controlled score by 15.71 percentage points.

However, the reached-mutant scores were much closer: 75.71% for the manual suite and 73.68% for the property-based suite. This shows that the main difference was behavioural reach rather than a large difference in fault-detection strength within executed code.

Survivor analysis identified genuine weaknesses in interest arithmetic, monetary rounding, interest scheduling, and positional form initialization. It also revealed equivalent mutants, underspecified diagnostic-message behaviour, and one survivor outside the principal scope.

The study therefore does not support the claim that manual testing is always superior to property-based testing. For this case study, the manual suite covered more behaviours, while the property-based suite provided comparable detection effectiveness within the behaviours it exercised.

The strongest conclusion is that the two techniques are complementary. Manual EP/BVA testing provides explicit requirement traceability and boundary coverage, while property-based testing provides broader input exploration and general invariant checking. Combining both approaches, supported by mutation analysis, offers a stronger verification strategy for financial software.

A Execution Evidence

A.1 Manual Test-Suite Evidence

The manual EP/BVA suite executed successfully with 29 passing test functions.

```
(venv) shibachowdhury@uc Srv-banking-verification-study % pytest -c nutnut_pytest.ini tests/manual -q
```

```
warnings summary [100%]
```

```
venv\Lib\python3.18/site-packages/django/apps/registry.py:91  
UserWarning: django.apps.registry.AppRegistry.unregister_app_hook() will be removed in Django 6.0.  
go now detects this configuration automatically. You can remove default_app_config =  
default_app_config = AppConfig.create_instance()
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html  
29 passed, 1 warning in 0.72s  
(venv) shibachowdhury@uc Srv-banking-verification-study %
```

Figure 4: Manual EP/BVA test-suite execution showing 29 passing tests.

A.2 Property-Based Test-Suite Evidence

The Hypothesis property-based suite executed successfully with 16 passing test functions.

```
(.venv) shihabchowdhury@Mac Srv-banking-verification-study % pytest -c mutmut_pytest.ini tests/property_based -q  
[.....] [100%]  
  
(.venv) Lib/python3.10/site-packages/django/apps/registry.py:91  
/Users/shihabchowdhury/Library/Courses/SW/Project/Srv-banking-verification-study/.venv/Lib/python3.10/site-packages/django/apps/registry.py:91: RemovedInDjango4Warning: 'django_celery_beat' defines default_app_config = 'django_celery_beat.apps.BeatConfig'. Django now detects this configuration automatically. You can remove default_app_config.  
app_config = AppConfig.create(entry)  
  
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html  
passed: 1 warning in 1.46s  
(.venv) shihabchowdhury@Mac Srv-banking-verification-study %
```

Figure 5: Property-based test-suite execution showing 16 passing tests.

A.3 Coverage Evidence

The manual suite achieved 84% overall structural coverage.

```

(.venv) shihabchowdhury@Mac Srv-banking-verification-study % pytest -c mutmut_pytest.ini tests/manual \
--covaccounts\models \
--covtransactions\forms \
--covtransactions\models \
--covtransactions\views \
--cov-branch \
--cov-report:term-missing

===== test session starts =====
platform darwin -- Python 3.10.20, pytest-9.1.1, pluggy-1.6.0
django: version: 3.2.5, settings: banking_system.settings (from ini)
rootdir: /Users/shihabchowdhury/Library/Courses/SW/Project/Srv-banking-verification-study
configfile: mutmut_pytest.ini
plugins: cov-7.2.0, celery-4.4.7, django-4.12.0, hypothesis-6.156.6
collected 29 items

tests/manual/test_account_model_behavior.py .....
tests/manual/test_deposit_validation.py ...
tests/manual/test_interestCalculation.py .....
tests/manual/test_transaction_processing.py .....
tests/manual/test_withdraw_validation.py .....

===== warnings summary =====
../venv/Lib/python3.10/site-packages/django/apps/registry.py:91
/Users/shihabchowdhury/Library/Courses/SW/Project/Srv-banking-verification-study/venv/Lib/python3.10/site-packages/django/apps/registry.py:91: RemovedInDjango4Warning: 'django_celery_beat' defines default_app_config = 'django_celery_beat.apps.BeatConfig' but
does not detect this configuration automatically. You can remove default_app_config.
app_config = AppConfig.create(entry)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

===== tests coverage =====
coverage: platform darwin, python 3.10.20-final-0

Name      Stmts  Miss Branch BrPart  Cover   Missing
src/accounts/models.py    55      5      2      0    90%   24, 124
src/transactions/forms.py  53     12     12      0    75%   86-93
src/transactions/models.py 11      0      0      0   100%
src/transactions/views.py  71     12      6      0    79%   24-28, 31-40, 43-49
TOTAL                192     26     20      0    84%

```

Figure 6: Manual-suite statement and branch coverage output.

The property-based suite achieved 77% overall structural coverage.

```
(.venv) shihabchowdhury@Mac Svv-banking-verification-study % pytest -c mutmut_pytest.ini tests/property_based \
--cov=accounts.models \
--cov=transactions.forms \
--cov=transactions.models \
--cov=transactions.views \
--cov=branch \
--cov-report=term-missing

platform darwin -- Python 3.10.20, pytest-9.1.1, pluggy-1.6.0
django: version: 3.2.9, settings: banking_system.settings (from ini)
rootdir: /Users/shihabchowdhury/University/Courses/SVV/Project/Svv-banking-verification-study
configfile: mutmut_pytest.ini
plugins: cov-7.1.0, celery-4.4.7, django-4.12.0, hypothesis-6.156.6
collected 16 items

tests/property_based/test_deposit_properties.py ... [ 10%]
tests/property_based/test_transaction_properties.py .. [ 31%]
tests/property_based/test_withdrawal_properties.py ..... [ 68%]
tests/property_based/test_interest_properties.py ..... [100%]

----- warnings summary -----
.venv/lib/python3.10/site-packages/django/apps/registry.py:91
/Users/shihabchowdhury/University/Courses/SVV/Project/Svv-banking-verification-study/.venv/lib/python3.10/site-packages/django/apps/registry.py:91: RemovedInDjango41Warning: 'django_celery_beat' defines default_app_config = 'django_celery_beat.apps.BeatConfig'. Django now detects this configuration automatically. You can remove default_app_config.
app_config = AppConfig.create(entry)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html

----- tests coverage -----
coverage: platform darwin, python 3.10.20-final-0

Name                               Stats    Miss Branch BrPart  Cover   Missing
-----
src/accounts/models.py              55      10      2      0    79%   24, 28-30, 51, 97, 105-109, 124
src/transactions/forms.py           53      12      12     0    75%   80-93
src/transactions/models.py          13       1       0     0    92%   27
src/transactions/views.py           71      15       6     0    75%   24-29, 31-40, 42-49, 66-71
TOTAL                               192      38      20     0    77%

16 passed, 1 warning in 1.97s
```

Figure 7: Property-based statement and branch coverage output.

A.4 Mutation-Testing Evidence

The manual mutation run produced 53 killed mutants, 17 surviving mutants, 26 no-test mutants, and no timeout mutants.

```
(.venv) shihabchowdhury@Mac Svv-banking-verification-study % cat results/mutation/manual/summary.txt
Manual PyTest Mutation Run

Total mutants: 96
Killed: 53
Survived: 17
No tests: 26
Timeout: 0
Suspicious: 0
Skipped: 0

Raw mutation score:
53 / 96 = 55.21%

Tested-mutant mutation score:
53 / (53 + 17) = 75.71%
```

Figure 8: Manual-suite mutation-testing summary.

The property-based mutation run produced 42 killed mutants, 15 surviving mutants, 39 no-test mutants, and no timeout mutants.

```
(.venv) shihabchowdhury@Mac Svv-banking-verification-study % cat results/mutation/property_based/summary.txt
Hypothesis Property-Based Mutation Run

Total mutants: 96
Killed: 42
Survived: 15
No tests: 39
Timeout: 0
Suspicious: 0
Skipped: 0

Out-of-scope mutants:
26 mutants in TransactionDateRangeForm.clean_daterange

Relevant in-scope mutants:
96 - 26 = 70

Relevant no-test mutants:
13
- UserBankAccount.__str__: 1
- UserBankAccount.get_interest_calculation_months: 12

Raw mutation score:
42 / 96 = 43.75%

Reached-mutant mutation score:
42 / (42 + 15) = 73.68%

Controlled in-scope mutation score:
42 / 70 = 60.00%
```

Figure 9: Property-based mutation-testing summary.

References

- [1] J. Hughes, “Software testing with quickcheck,” in *Central European Functional Programming School: Third Summer School, CEFPS 2009, Revised Selected Lectures*, ser. Lecture Notes in Computer Science, Z. Horváth, R. Plasmeijer, and V. Zsók, Eds. Berlin, Heidelberg: Springer, 2010, vol. 6299, pp. 183–223.
- [2] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
- [3] M. Papadakis, S. Yoo, D. Shin, and D.-H. Bae, “Are mutation scores correlated with real fault detection? a large-scale empirical study on the relationship between mutants and real faults,” in *Proceedings of the 40th International Conference on Software Engineering*. Association for Computing Machinery, 2018, pp. 537–548.
- [4] S. Ravi and M. Coblenz, “An empirical evaluation of property-based testing in python,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 3897–3923, Oct. 2025.

- [5] saadm11, “Banking system: A banking system concept created using the django python web framework and tailwind css,” <https://github.com/saadm11/banking-system>, gitHub repository; evaluated revision 9c3ee3eea75030280405506c62d968d646981c05; accessed July 2026.